iEMS-py

Architecture and operation of a real-time energy management strategy for a fuel cell hybrid vehicle

Version 1.0 — milestone M1

Matthieu Matignon — repository `Matti-D-Rocketta/iems-py`

All figures are generated by `scripts/make_doc_figures.py`; all numbers come from `docs/m1_report.md`.

Contents

1 Executive summary 4
2 The vehicle and the energy problem 5
2.1 The powertrain 5
2.2 Why the split matters 5
2.3 What "charge-sustaining" means 5
2.4 The driving cycles 6
3 Three strategies compared 7
3.1 The strategies 7
3.2 Results on five cycles 7
3.3 Reading the thesis-style figure 8
3.4 Sensitivity 8
4 Software architecture 10
4.1 Module map 10
4.2 One EMS step 11
4.3 Two sampling rates 12
4.4 Repository layout 12
4.5 "I want to change X" → "I edit Y" 12
5 Physical models 14
5.1 Vehicle — models/vehicle.py 14
5.2 Fuel cell system — models/fuel_cell.py 14
5.3 Battery — models/battery.py 15
5.4 Driving cycles — models/cycles.py 16
6 The iEMS modules, one by one 17
6.1 Speed predictor — dpt/predictor.py 17
6.2 Driving pattern recognition — dpt/dpr.py 17
6.3 Candidates — candidates.py 18
6.4 Balance — balance.py 19
6.5 Optimizer — optimizer.py 20
6.6 Strategies — strategies.py 20
6.7 Simulator — simulator.py 21
6.8 Metrics — metrics.py 21
7 Verification and validation 22
7.1 Automated tests 22
7.2 Acceptance criteria and the reference report 22
7.3 How to judge a change 23

7.4 Known limits of version 1 23
8 Using the package 24
8.1 Installation 24
8.2 The three scripts 24
8.3 The Python API 24
8.4 Typical experiments 24
9 Roadmap, glossary, references 25
9.1 Roadmap 25
9.2 Glossary 25
9.3 References 26

1 Executive summary

In two sentences. A fuel cell hybrid vehicle has two power sources, a hydrogen fuel cell and a battery, and must decide at every instant how much each one delivers. iEMS-py is a Python simulator in which an *integrated energy management strategy* (iEMS) makes that decision in real time, and is compared against two reference strategies on standard and synthetic driving cycles.

The problem. Splitting the power demand between the fuel cell and the battery is a trade-off made every second. The fuel cell is efficient only in part of its operating range and cannot change its output quickly; the battery reacts instantly but has a limited energy reserve and must end the trip roughly where it started (*charge-sustaining*). A good energy management strategy (EMS) minimises hydrogen consumption while respecting all physical limits and without knowing the future of the trip.

The solution. The iEMS combines three families of methods from the author's doctoral work: optimisation (Pontryagin's minimum principle applied online, with a co-state adapted in real time), expert rules (a battery usage target that depends on speed, state of charge and driving pattern) and driving pattern recognition (traffic, urban, main road, highway, sporty). The whole package is a clean-room Python rewrite: no file from the original industrial Simulink project is reused.

What has been shown (milestone M1). On the WLTC class 3b cycle and four labelled synthetic cycles, with default parameters:

Result	Value
Hydrogen consumption on WLTC, SOC-corrected	iEMS **817.0 g/100 km**, rule-based baseline 821.6, offline-calibrated ECMS 803.8
iEMS versus baseline	lower consumption on 5 cycles out of 5 (−0.6 % to −1.4 %)
iEMS versus offline reference (knows the whole cycle)	within +1.6 % on the worst cycle
Charge-sustaining on WLTC	ΔSOC = −0.55 points without any knowledge of the end of the cycle
Constraint violations	0 on 5 cycles × 3 strategies
Energy balance of the DC bus	closed to better than 1e-9 %
Adaptation to the initial state of charge	starting at 35 % or 75 %, the iEMS ends WLTC at the same ≈ 60 %
Verification	358 automated tests, 6 quantified acceptance criteria, regenerable report

How to read this document. Chapters 1 and 2 need no technical background. Chapter 3 gives the software architecture with three diagrams. Chapters 4 and 5 go module by module, with equations, parameters, design decisions and the test file that protects each one. Chapter 6 explains how the software is verified and how to judge any change. Chapter 7 shows how to use it. Chapter 8 gives the roadmap, a glossary and references.

2 The vehicle and the energy problem

In two sentences. The vehicle is driven by an electric motor fed by a DC bus; the bus is fed by a fuel cell and a battery. The EMS decides, every second, the fuel cell power — the battery supplies or absorbs the rest.

2.1 The powertrain

Figure 1 shows the energy flows. Hydrogen from the tank feeds the fuel cell system, which delivers electrical power `P_fc` to the DC bus. The battery exchanges power `P_bat` with the bus, positive when discharging. The inverter and motor draw the traction power from the bus, or push power back into it when the vehicle brakes (regenerative braking). Auxiliaries (climate, electronics) draw a constant `P_aux`.

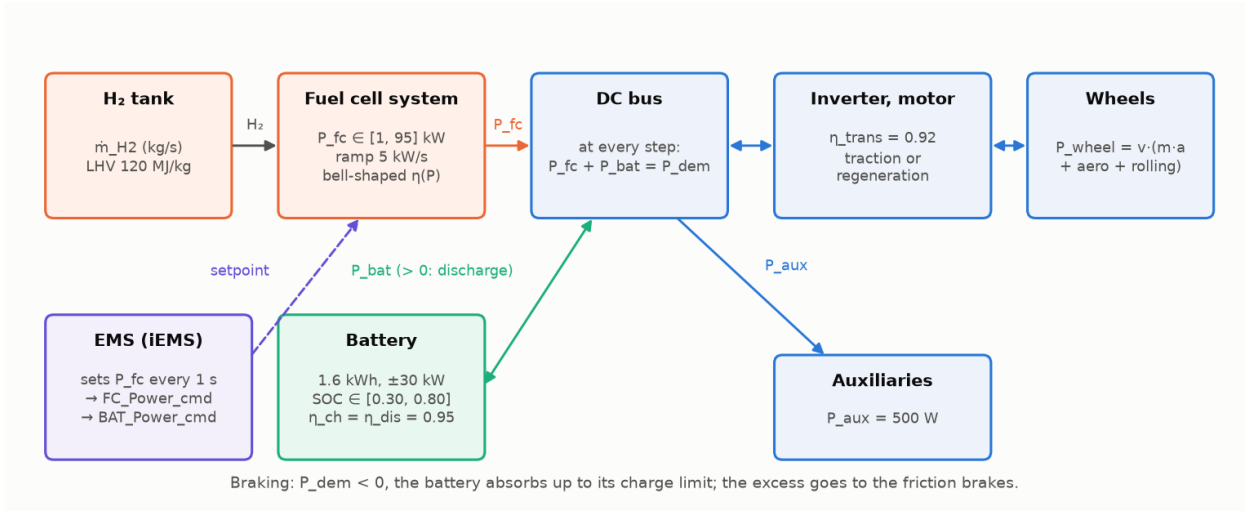


Figure 1 — Powertrain of the fuel cell hybrid vehicle and energy flows. The EMS sets the fuel cell power; the DC bus balance then fixes the battery power.

At every instant the bus must be balanced: `P_fc + P_bat = P_dem`, where `P_dem` is the electrical demand (traction plus auxiliaries). This single equation is the heart of the problem: once the EMS has chosen `P_fc`, the battery has no choice.

2.2 Why the split matters

Three physical facts make the decision non-trivial:

- **The fuel cell has a preferred operating range.** Its efficiency is low at idle (the compressor and auxiliaries consume hydrogen even when little power is produced), peaks around 20 % of its maximum power (≈ 19 kW for the 95 kW system modelled here) and decreases at high load. Running the fuel cell near its efficient range saves hydrogen.
- **The fuel cell is slow.** Its power can only change by 5 kW per second. Sharp accelerations must be covered by the battery, which means the fuel cell has to be *pre-positioned* before a known demand peak.
- **The battery is a buffer, not a source.** Over a trip, what is taken from the battery must be given back by the fuel cell (or by regenerative braking). The state of charge (SOC) must stay inside a usable window (30 % to 80 % here) and end the trip near its starting value, otherwise the consumption figure is meaningless.

2.3 What "charge-sustaining" means

A strategy is charge-sustaining when the SOC at the end of a representative trip is close to the SOC at the start. Two strategies can only be compared fairly if both are charge-sustaining, or if their consumption is

corrected for the energy they borrowed from (or stored into) the battery. This document reports *SOC-corrected* consumption: the energy difference $\Delta E = (SOC_0 - SOC_T) \cdot E_cap$ is converted into the hydrogen it would cost (or save) at the average fuel cell efficiency of the run (`src/iems/metrics.py`).

2.4 The driving cycles

The vehicle is simulated on speed profiles over time. Five cycles are used (chapter 4.4): a structural reconstruction of the WLTC class 3b homologation cycle (23.27 km, 1800 s) and four synthetic cycles generated by concatenating typed segments (urban, traffic, main road, highway, sporty). Each synthetic sample carries the label of the segment it belongs to, which gives a ground truth for the driving pattern recogniser.

3 Three strategies compared

In two sentences. Three strategies share the same interface and the same physical models, so that only the decision logic differs. The iEMS is compared with a simple rule-based controller and with an offline-optimal reference that is allowed to know the whole cycle in advance.

3.1 The strategies

Strategy	Principle	Role
`rules_baseline`	*Power follower*: the fuel cell tracks the low-pass-filtered demand (5 s time constant); the battery absorbs the rest. A SOC hysteresis adds +8 kW of fuel cell power when the SOC drops below 50 %, and sheds load above 62 %.	What a reasonable engineer writes in an afternoon. The bar to beat.
`ecms_const`	Equivalent Consumption Minimisation Strategy with a *constant* equivalence factor λ, calibrated by bisection so that the SOC ends where it started. Calibrated with an exact prediction of the cycle.	Quasi-optimal offline reference. It knows the future; a real-time strategy cannot.
`iems`	The full pipeline: driving pattern recognition, noisy speed prediction over a 10 s horizon, admissible candidate set, battery usage rules, adaptive co-state, Hamiltonian minimisation.	The strategy under study.

All three run on the same simulator at the same two sampling rates, pass through the same admissibility block (so the comparison is about the energy decision, not about who respects the constraints) and are measured by the same metrics.

3.2 Results on five cycles

Cycle	km	Rules (g/100 km)	ECMS offline	iEMS	ΔSOC iEMS (pts)	Violations
WLTC class 3b	23.27	821.6	803.8	**817.0**	−0.55	0
Synthetic mixed	30.61	832.3	824.2	**823.9**	+1.17	0
Dense urban	6.99	969.8	982.7	**956.3**	+2.64	0
Main road + highway	47.89	936.4	922.5	**928.2**	−1.09	0
Sporty	21.18	890.9	883.1	**880.8**	+0.09	0

Consumptions are SOC-corrected. The iEMS beats the baseline on every cycle and stays within 1.6 % of the offline reference on the worst cycle (WLTC). On the dense urban cycle the iEMS even beats the offline ECMS: with a constant λ the reference keeps the fuel cell at its inefficient idle for most of a low-demand cycle, whereas the adaptive co-state lets the iEMS cycle the fuel cell through its efficient range.

The rule-based baseline is not charge-sustaining (ΔSOC = +10.1 points on WLTC): it ends with a fuller battery, which the correction charges back to it. Without the correction its raw consumption would look worse (861.7 g/100 km on WLTC versus 814.6 for the iEMS).

3.3 Reading the thesis-style figure

Figure 2 is the reference view used throughout the project: five panels sharing the time axis.

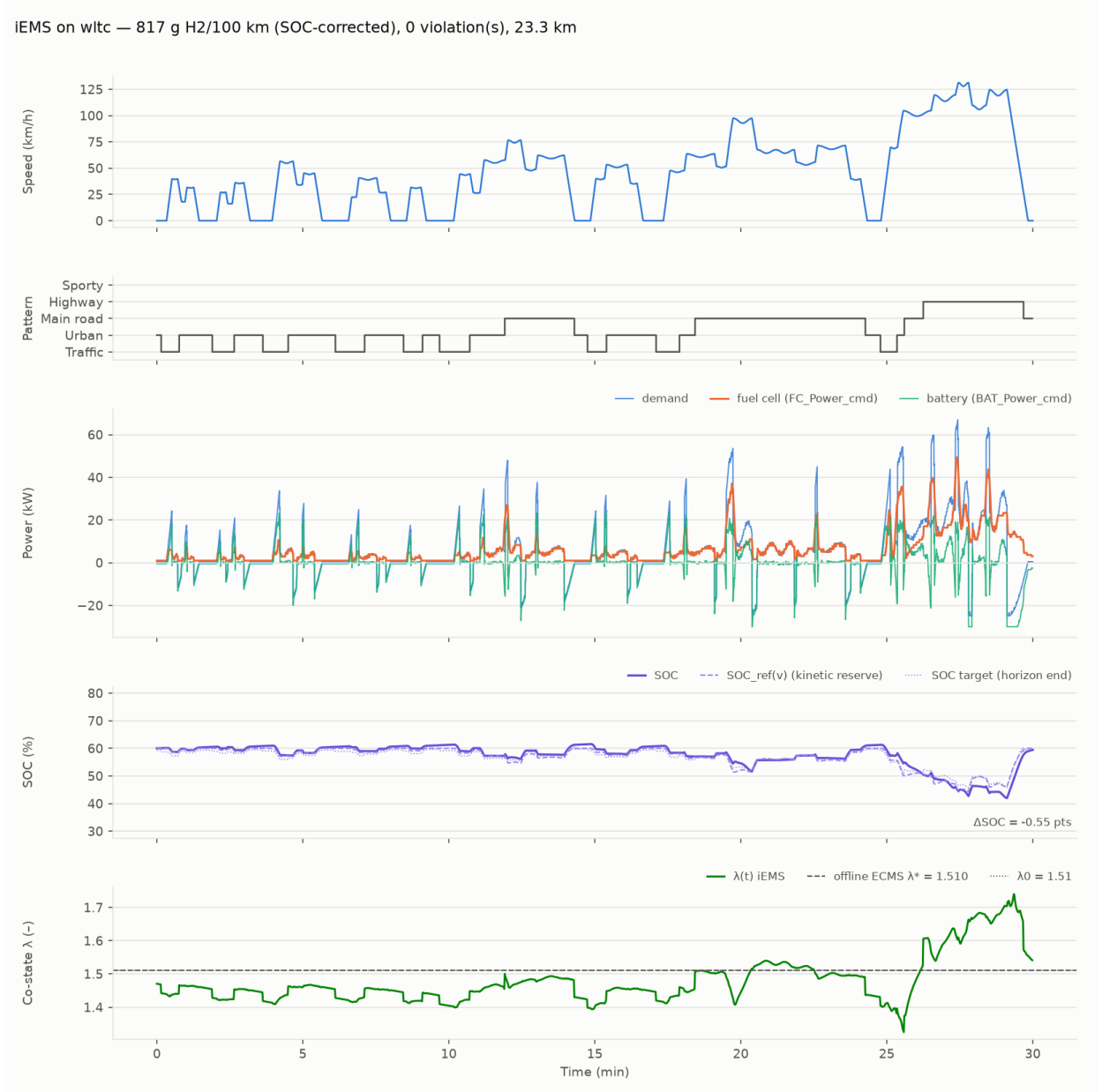


Figure 2 — iEMS on WLTC: speed, recognised driving pattern, power split, SOC with its reference and target, co-state λ.

- **Powers.** The fuel cell (orange) is smooth, the battery (green) takes the peaks: high-frequency content goes to the battery, low-frequency content to the fuel cell, as in the thesis results.
- **SOC.** The dashed line `SOC_ref(v)` drops when the speed rises: it is the *kinetic reserve* (chapter 5.4), the room left in the battery for the energy that will come back at the next braking. The SOC follows it with some lag; the final braking from 125 km/h brings it back to 60 %.
- **Co-state λ.** It oscillates around the offline-optimal value λ* = 1.510 (dashed), another qualitative result of the thesis reproduced here: λ rises when the SOC is below its target (battery "expensive", the fuel cell recharges it) and falls above.

3.4 Sensitivity

The sensitivity table of the M1 report (`docs/m1_report.md`) varies what could break the iEMS on WLTC:

Case	H_2 corrected (g/100 km)	ΔSOC (pts)	Violations
Exact prediction	817.2	−0.37	0
Default prediction noise (seed 42)	817.0	−0.55	0
Noise ×2	816.9	−0.35	0
Noise ×3	823.5	+0.12	55
Other noise seeds (1, 2, 3)	816.3 / 816.4 / 816.9	−0.30 / +0.18 / −0.34	0
Initial SOC 35 %	802.6	+24.45 (ends at 59.5 %)	0
Initial SOC 75 %	841.0	−15.55 (ends at 59.5 %)	0

The strategy holds up to twice the default prediction error; at three times, violations reappear (the admissibility margins are sized for the default error level, chapter 5.3). Starting from 35 % or 75 %, the iEMS ends the cycle at the same SOC: the adaptation to the initial state of charge is a built-in property of the co-state loop, not a tuning.

4 Software architecture

In two sentences. The package is organised around one EMS step: four blocks (pattern and prediction, candidates, balance, optimiser) compute a fuel cell setpoint every second, and a simulator applies it at a finer time step while integrating the physics. Every module has a single responsibility, a documented interface and its own test file.

4.1 Module map

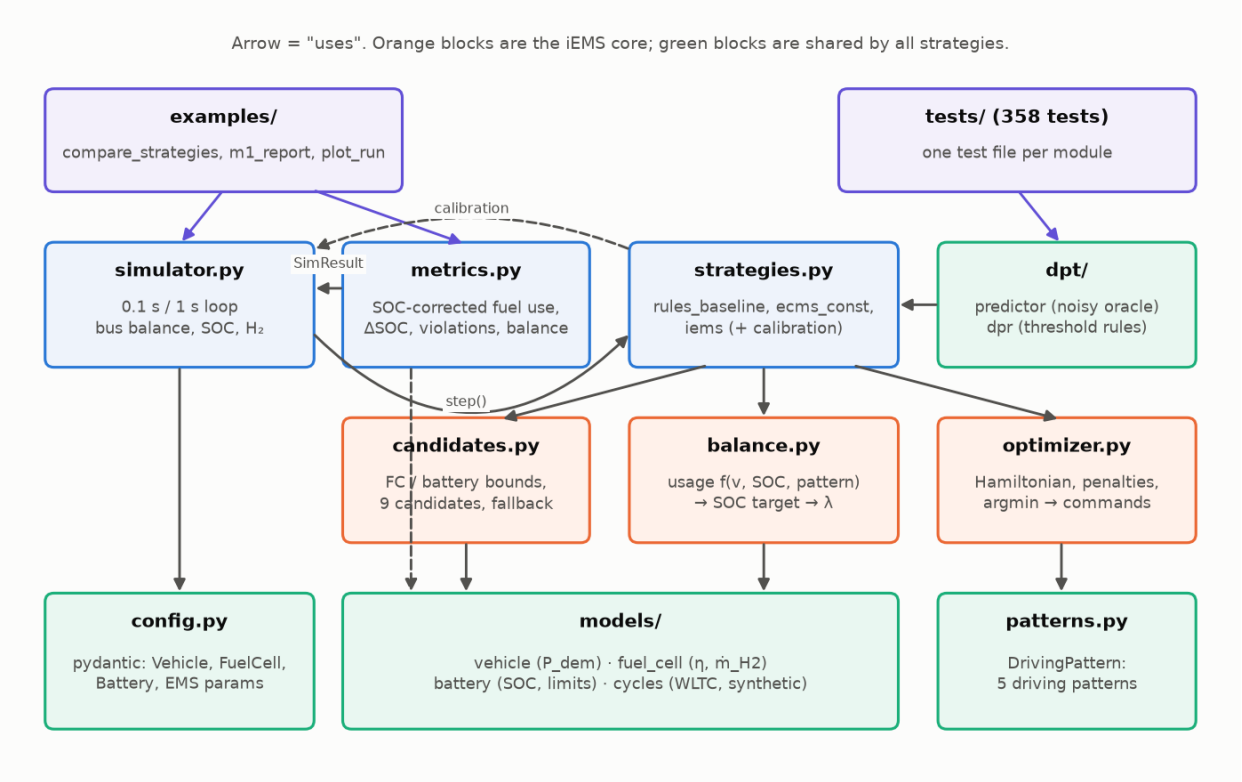


Figure 3 — Module map of the package. An arrow means "uses". Orange blocks are the iEMS core; green blocks are shared by all strategies.

Module	Responsibility	Tests
`config.py`	All parameters, as frozen pydantic models with validation; each constant tagged with its source.	`test_models.py` (validation cases)
`models/vehicle.py`	Electrical demand from a speed profile (longitudinal equation).	`test_models.py`
`models/fuel_cell.py`	Efficiency map, convex hydrogen consumption, bounds and ramp.	`test_models.py`
`models/battery.py`	Energy-based SOC model, derated charge/discharge limits.	`test_models.py`
`models/cycles.py`	Embedded WLTC, labelled synthetic cycle generator, resampling.	`test_models.py`
`dpt/predictor.py`	Speed prediction over the horizon (v1: noisy oracle).	`test_dpt.py`
`dpt/dpr.py`	Driving pattern recognition (v1: threshold rules).	`test_dpt.py`
`candidates.py`	Fuel cell and battery bounds, anticipation over the horizon, 9	`test_candidates.py`

	candidates, fallback vector.	
`balance.py`	Battery usage table, SOC target, co-state λ.	`test_balance_optimizer.py`
`optimizer.py`	Hamiltonian per candidate, penalties (pass-through), argmin → commands.	`test_balance_optimizer.py`
`strategies.py`	The three strategies behind one interface; ECMS calibration by bisection.	`test_strategies_metrics.py`
`simulator.py`	Fixed-step loop at two rates, bus balance check, SOC and H_2 integration, logs.	`test_simulator.py`
`metrics.py`	Consumption (raw and SOC-corrected), ΔSOC, violation counters, energy balance.	`test_strategies_metrics.py`

4.2 One EMS step

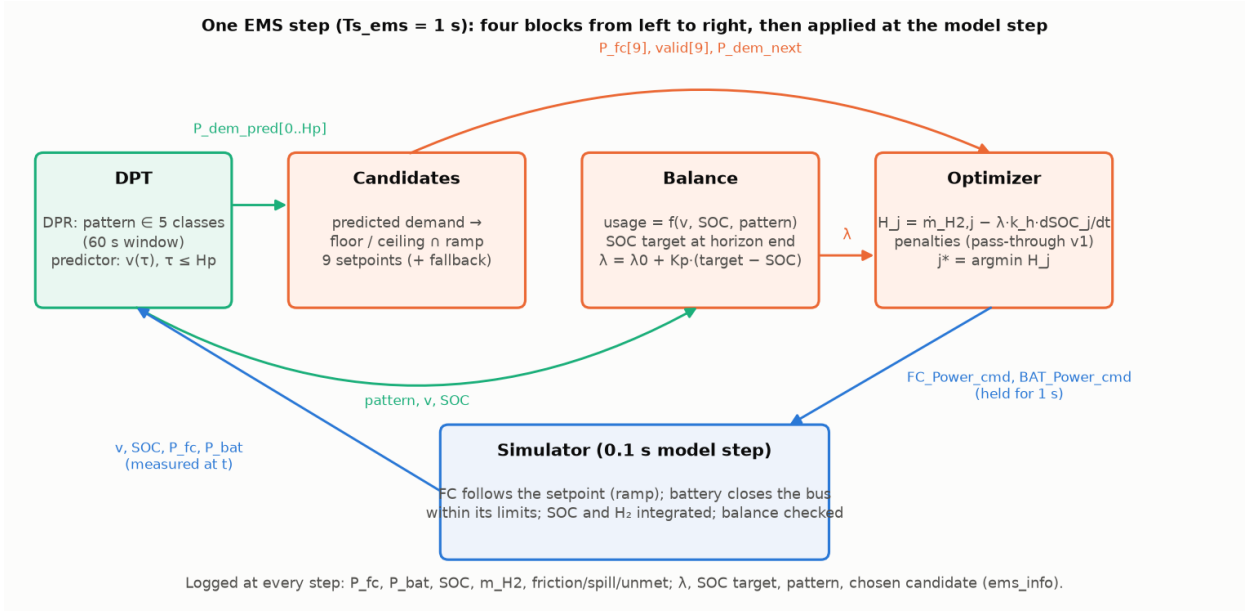


Figure 4 — One EMS step. Signals exchanged between the four blocks, then application by the simulator at the model step.

Every `Ts_ems = 1 s`, the strategy receives an observation (speed, SOC, applied fuel cell and battery powers, current demand, predicted demand over the horizon, speed history) and returns two commands, `FC_Power_cmd` and `BAT_Power_cmd`, with a dictionary of diagnostics. For the iEMS the step is:

1. **DPT** — the recogniser classifies the last 60 s of speed into one of five patterns; the predictor returns the speed over the next `Hp = 10 s`, from which the predicted electrical demand is computed.
2. **Candidates** — from the fuel cell ramp and the predicted demand, the block computes the usable range `[P_lo, P_hi]` for the next setpoint and builds nine candidates inside it (or a fallback vector when the range collapses).
3. **Balance** — a battery usage ratio is read from a table indexed by pattern, speed and SOC; it is turned into a SOC target at the end of the horizon, then into the co-state `λ = λ0 + Kp·(SOC_target − SOC)`.
4. **Optimizer** — a Hamiltonian is evaluated for each candidate, `H_j = ṁ_H2,j − λ·k_h·dSOC_j/dt`; penalties are a pass-through in v1; the candidate with the lowest Hamiltonian becomes `FC_Power_cmd`, and `BAT_Power_cmd = P_dem − FC_Power_cmd`.

The names `FC_Power_cmd` and `BAT_Power_cmd` are kept from the original design for continuity.

4.3 Two sampling rates

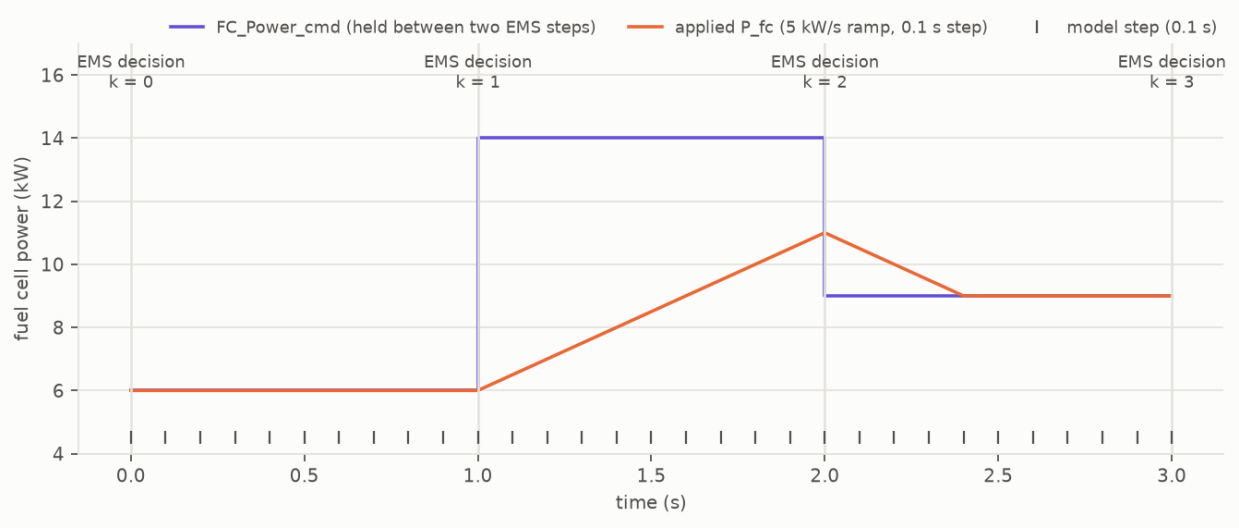


Figure 5 — The two sampling rates. The EMS decides every second; the command is held; the model applies it every 0.1 s with the fuel cell ramp.

The physical model runs at `Ts_model = 0.1 s`; the EMS runs at `Ts_ems = 1 s`, which must be an integer multiple (validated in `config.py`). Between two EMS decisions the command is held (zero-order hold), and the fuel cell power follows it under its 5 kW/s ramp. This explicit rate transition is one of the known pitfalls inherited from the original design; it is tested in `test_simulator.py` (command held, ramp never exceeded, one EMS step per `substeps` model steps).

4.4 Repository layout

```
iems-py/
├── pyproject.toml          # package metadata, dependencies, ruff and pytest settings
├── README.md               # pitch, quickstart, results, architecture
├── NOTES_REPRISE.md        # hand-over notes between milestones
├── src/iems/               # the package (see module map)
│   ├── data/wltc_class3.csv
│   ├── models/  dpt/
│   └── *.py
├── tests/                  # 358 pytest tests, one file per module group
├── examples/               # compare_strategies.py, m1_report.py, plot_run.py
├── scripts/                # build_wltc_csv.py, make_doc_figures.py, build_doc.py
└── docs/                   # reference report, figures, this document
```

4.5 "I want to change X" → "I edit Y"

I want to…	File	Entry point
change a vehicle, fuel cell, battery or EMS parameter	`config.py`	the four `*Params` classes
try another battery physics (internal resistance, v2)	`models/battery.py`	`Battery.step`, `soc_rate`, limits
use another fuel cell efficiency map	`config.py` then `models/fuel_cell.py`	`eta_map_*`, `fit_consumption`
change the admissibility logic or the number of candidates	`candidates.py`	`anticipation_bounds`, `build_candidates`
change the battery usage rules or the SOC target	`balance.py`	`build_usage_table`, `Balance.step`
activate penalties in the Hamiltonian	`optimizer.py`	`apply_penalties` (v2 hook)

add a strategy	`strategies.py`	a class with `.step(obs)`, registered in `STRATEGIES`
plug a real predictor (LSTM)	`dpt/predictor.py`	a class implementing `SpeedPredictor`
plug a fuzzy pattern recogniser	`dpt/dpr.py`	same `recognize(v_hist)` interface
add a metric	`metrics.py`	`compute_metrics`, `Metrics.row`
add a driving cycle	`models/cycles.py`	`get_cycle`, the catalogue

Rule of thumb: every line of this table has a test file of the same name; run it first after a change.

5 Physical models

In two sentences. The models are deliberately simple (energy-based, static maps), documented with their equations and default values, and every numerical assumption carries a source tag. Default parameters are generic for a sedan-class fuel cell hybrid and are marked `# TODO-thèse` where they should be replaced by the thesis values.

5.1 Vehicle — models/vehicle.py

The electrical demand is computed from the speed profile with the longitudinal equation on a flat road:

`P_wheel = v · (m·a + ½·ρ·Cd·Af·v² + Crr·m·g)`

`P_dem = P_wheel / η_trans + P_aux` in traction, `P_dem = P_wheel · η_trans + P_aux` in braking (the drivetrain loses energy in both directions).

Parameter	Default	Source
mass m	1800 kg	TODO-thèse
Cd, Af	0.29, 2.4 m²	TODO-thèse
Crr	0.009	TODO-thèse
η_trans	0.92	TODO-thèse
P_aux	500 W	TODO-thèse
ρ_air, g	1.2 kg/m³, 9.81 m/s²	literature

Negative demand (braking) is absorbed by the battery up to its charge limit; the excess is dissipated in the friction brakes and logged as `P_friction`, never into the SOC (pitfall 3 of the specification).

5.2 Fuel cell system — models/fuel_cell.py

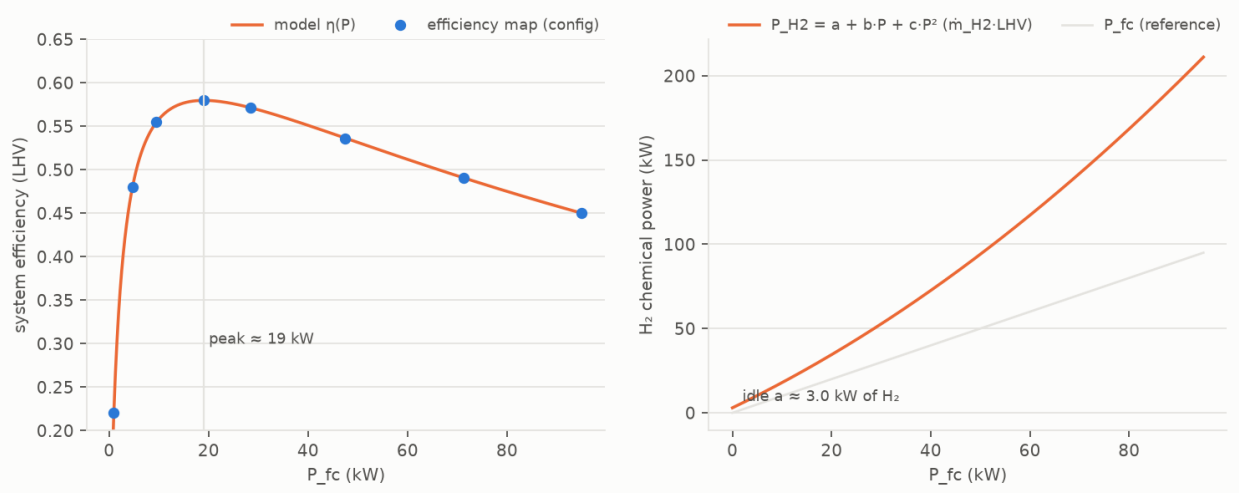


Figure 6 — Fuel cell model: bell-shaped efficiency map and the convex consumption curve fitted on it.

The configuration gives a bell-shaped efficiency map: ≈ 0.40 at 3 % load, peak 0.58 around 20 % of `P_max`, 0.45 at full load. Rather than interpolating η linearly, the model fits a **convex consumption curve** on that map:

`P_H2(P) = ṁ_H2 · LHV = a + b·P + c·P²` with a ≈ 2.97 kW (idle hydrogen power), c > 0.

The efficiency is then `η(P) = P / P_H2(P)`, which reproduces the bell (peak at √(a/c) ≈ 19 kW). This choice is the most important modelling decision of the project: with a linearly interpolated η, the cost `P/η(P)` is concave at low load and linear on the plateau, the Hamiltonian has no interior minimum, the control becomes

bang-bang and the calibration of λ is ill-posed (ΔSOC jumps from −16 to +20 points for Δλ = 0.2). The convex curve makes the Hamiltonian strictly convex.

Parameter	Default	Source
P_max, P_min	95 kW, 1 kW (idle; v1 never switches the stack off)	TODO-thèse
ramp r_fc	5 kW/s	TODO-thèse
efficiency map	8 points, 0.40 → 0.58 → 0.45	literature shape, TODO-thèse values
LHV of hydrogen	120 MJ/kg	literature

5.3 Battery — models/battery.py

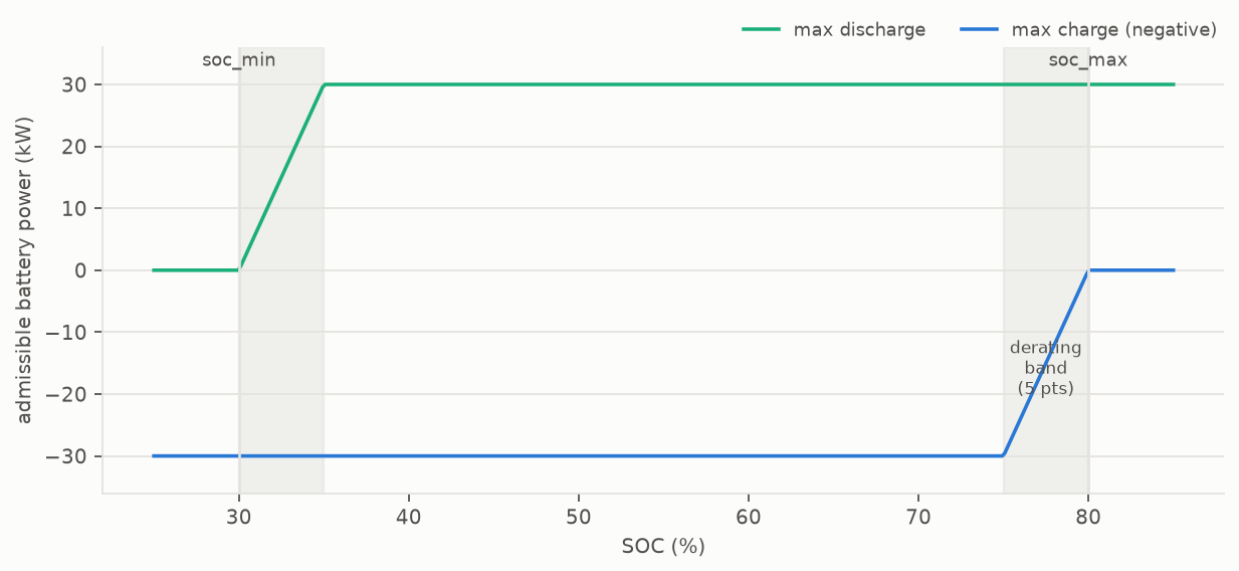


Figure 7 — Battery limits versus SOC: admissible discharge and charge power are derated linearly to zero over a 5-point band at each end of the usable window.

Version 1 is an energy model with constant efficiencies:

`dSOC/dt = −P_int / E_cap` with `P_int = P_bat / η_dis` when discharging, `P_bat · η_ch` when charging (`E_cap` in joules).

The BMS limits are derated: the admissible discharge power decreases linearly to zero over a 5-point band above `soc_min`, and symmetrically for charge below `soc_max`. These soft ramps guarantee that the SOC never leaves the usable window, which is verified in `test_simulator.py` on every cycle.

Parameter	Default	Source
E_cap	1.6 kWh	TODO-thèse
P_dis_max, P_ch_max	30 kW, 30 kW	TODO-thèse
usable window	[0.30, 0.80], derating band 0.05	TODO-thèse
η_ch, η_dis	0.95, 0.95	TODO-thèse

A version 2 with an internal resistance model (`I = (U_oc − √(U_oc² − 4·R·P)) / (2R)`) is planned behind the same interface.

5.4 Driving cycles — models/cycles.py

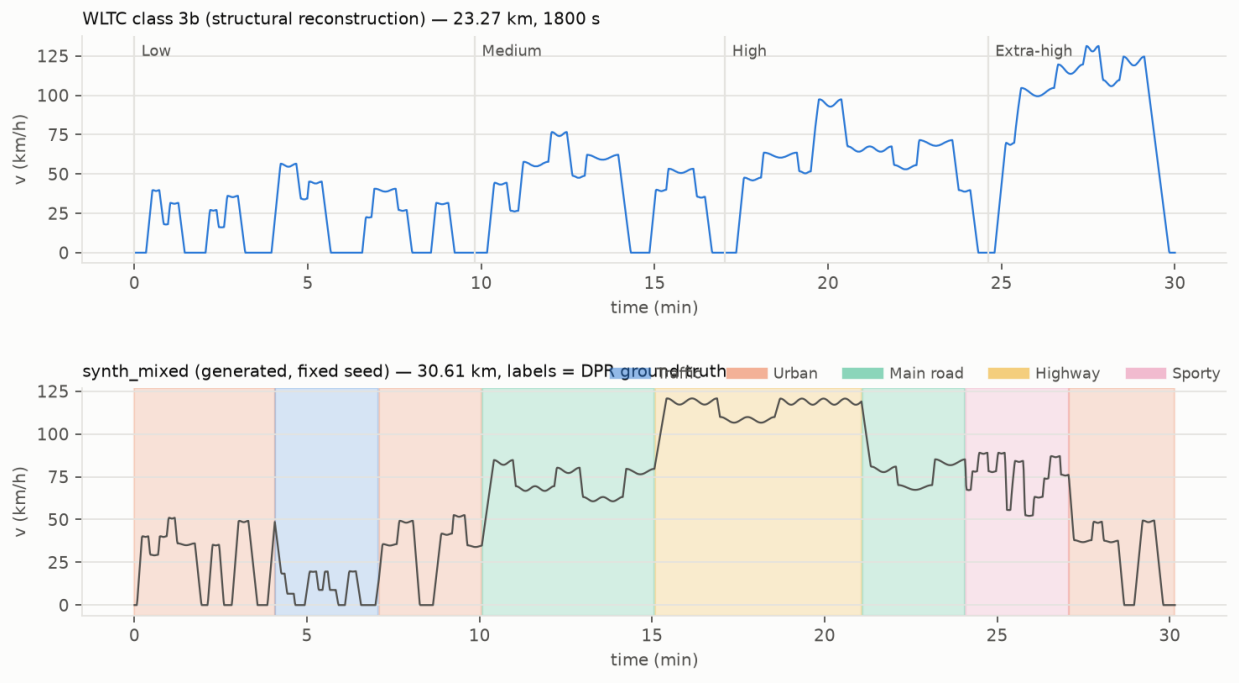


Figure 8 — The embedded WLTC reconstruction (top) and a labelled synthetic cycle (bottom). Coloured bands are the ground-truth driving patterns.

A cycle is a `Cycle(name, t, v)` object (time in s, speed in m/s), optionally with per-sample labels. Two sources:

- **WLTC class 3b** is embedded as `src/iems/data/wltc_class3.csv`. It is a *structural reconstruction*: the four phases have the official durations (589/433/455/323 s), distances (3095/4756/7162/8254 m) and maximum speeds (56.5/76.6/97.4/131.3 km/h), with stops at phase ends, but it is not the official point-by-point table. Dropping the official table at the same path (columns `t_s,v_kmh`) replaces it without any other change. Resampling to the model step is linear, as the WLTP procedure prescribes; a cubic interpolation would inflate acceleration peaks.
- **Synthetic cycles** are generated by concatenating typed segments (traffic, urban, main road, highway, sporty), each built from micro-trips with jerk-limited accelerations and power-capped acceleration rates. Every sample carries its segment label: this is the ground truth for the pattern recogniser. The generator is deterministic for a given seed. Four cycles are in the catalogue: mixed (all five patterns), dense urban, main road + highway, sporty.

Adding a cycle from a CSV (speed over time) is a matter of building a `Cycle` and registering it in `get_cycle`; this is also the ingestion step of milestone M4.

6 The iEMS modules, one by one

In two sentences. Each block has a single job, a documented interface and a few physically motivated parameters. The design decisions below are the ones to be able to defend: the inertial prediction margin, the kinetic reserve, the homogenisation factor of the Hamiltonian and the second-order error model of the predictor.

6.1 Speed predictor — dpt/predictor.py

Role. Return the speed over the next `Hp = 10 s` at the model step, from which the predicted electrical demand is computed. Version 1 is a *noisy oracle*: it reads the true future of the cycle and distorts it. Version 2 (LSTM, optional `torch` extra) will implement the same `SpeedPredictor` protocol: `reset()` and `predict(i) → array of n_h + 1 speeds`.

Error model. Two distortions are drawn per call, each following an AR(1) process between calls (ρ = 0.95) so that the error is persistent, as a real predictor's is:

- a *timing* error, `τ' = τ + β·τ²/T_ref` (an expected acceleration arrives early or late);
- an *amplitude* error, a multiplicative bias `γ·(τ/T_ref)²`.

Both are **second order in τ**: they vanish in value *and in slope* at the current instant. This is the physical property of an embedded predictor: the current speed and acceleration are measured (torque request), only the future is uncertain. A linear bias in τ would impose a phantom acceleration `γ·v/T_ref` from τ = 0, i.e. ±20 kW at 130 km/h for a 9 % end-of-horizon error. The reference horizon `T_ref = 10 s` makes the error at τ seconds independent of the requested horizon length.

Parameter	Default	Meaning
`predictor_noise`	0.03	standard deviation of the amplitude bias at τ = 10 s
`predictor_time_warp`	0.03	standard deviation of the timing error at τ = 10 s
`seed`	42	fixed seed, deterministic runs

Tests (`tests/test_dpt.py`): exact value and slope at τ = 0, length and edge padding at the end of the cycle, determinism per seed, end-of-horizon error magnitude.

6.2 Driving pattern recognition — dpt/dpr.py

Role. Classify the recent driving into one of five patterns (traffic, urban, main road, highway, sporty) from features of a sliding window of speed (60 s): mean and maximum speed, standard deviation, mean positive acceleration, 95th percentile of |a|, share of time at standstill.

Algorithm (v1). Threshold rules, from the most discriminating to the most generic: sporty if the 95th percentile of |a| exceeds 1.3 m/s²; highway if the mean speed exceeds 95 km/h; main road above 50 km/h; traffic if the maximum speed is below 30 km/h or the standstill share exceeds 45 %; urban otherwise. The thresholds were tuned by a grid search on the four labelled synthetic cycles (mean accuracy 0.80 → 0.87, sporty recall 0.47 → 0.85). Version 2 (fuzzy c-means, optional `scikit-fuzzy` extra) keeps the `recognize(v_hist)` interface; the thesis class centres are the `TODO-thèse` values.

Tests (`tests/test_dpt.py`): features on constant and zero speed, each class reachable by hand-built features, priority of rules, accuracy ≥ 0.85 on the urban and highway cycles.

6.3 Candidates — candidates.py

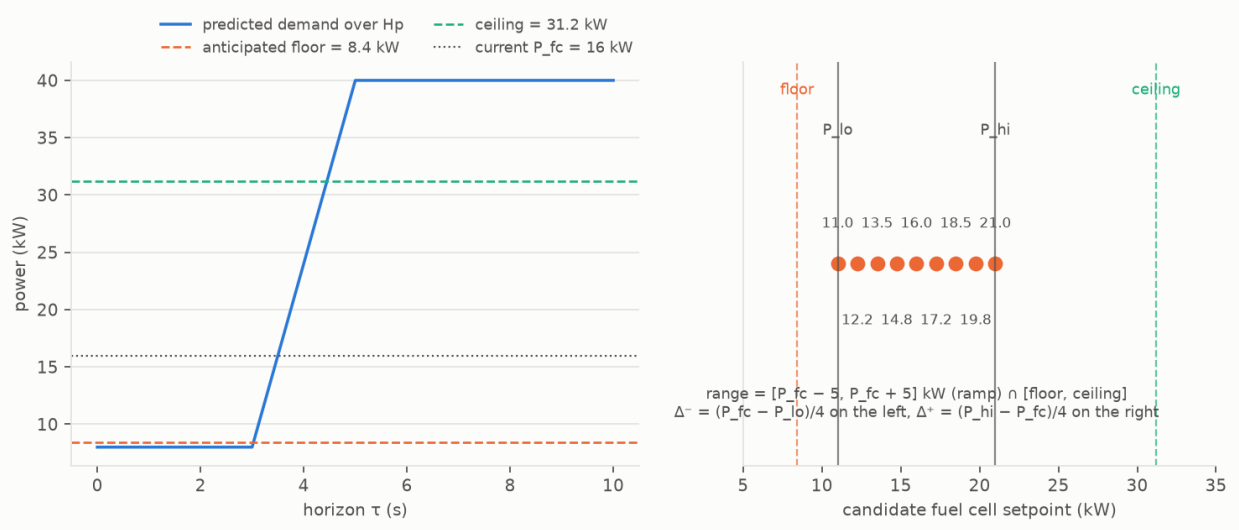


Figure 9 — Building the 9 candidates on a real case: a predicted ramp from 8 to 40 kW three seconds ahead, fuel cell currently at 16 kW. Left: the anticipated floor and ceiling. Right: the usable range and the candidates.

Role. Compute the range of fuel cell setpoints that the system can actually follow over the next second *and* keep followable over the horizon, then offer nine candidates in that range.

Algorithm. Three layers of limits:

5. **Fuel cell**: static bounds `[P_min, P_max]` and the ramp give `[P_lo, P_hi] = [P_fc − r·Ts, P_fc + r·Ts]`.
6. **Battery over the horizon**: on every interval of the predicted demand, the battery must be able to close the bus, `P_bat = P_dem − P_fc`, within its (derated) limits. Because the fuel cell can only move by `r·Ts` per step, today's setpoint inherits an anticipated **floor** (be high enough before a demand peak) and **ceiling** (be low enough before a demand drop). The demand extrema are dilated by ±1 interval to be robust to a one-second timing error.
7. **Synthesis**: intersection of the two; when the floor exceeds the ceiling, the prediction margins are eroded by bisection until a range exists; if it is impossible even without margins, the midpoint is taken and the step is flagged as a conflict.

The nine candidates are `[P_lo, c − 3Δ⁻, c − 2Δ⁻, c − Δ⁻, c, c + Δ⁺, c + 2Δ⁺, c + 3Δ⁺, P_hi]` with `c` the current power brought into the range, `Δ⁻ = (c − P_lo)/4` and `Δ⁺ = (P_hi − c)/4` — the quarter-step construction of the original design. When the range is degenerate (bounds within 1 W), the **fallback vector** holds nine copies of the single admissible value: never a NaN (pitfall 2).

The inertial margin. The electrical demand depends on the acceleration, i.e. on the derivative of the predicted speed. At 110 km/h, an error of 1 km/h on the speed predicted one second ahead is worth ≈ 15 kW of demand (`m·v·Δa/η`). The margin applied to the predicted demand is therefore `prediction_margin·|P| + m·v·σ_a/η_trans`, with `σ_a = 0.10 m/s²`: small in town, large on the highway, where the error is too. Battery limits are additionally reduced by 5 % and 1 kW and evaluated at the worst SOC reachable within one step.

Tests (`tests/test_candidates.py`): exactly nine ordered candidates with the quarter spacing, fallback without NaN on four degenerate constructions, interval extrema and dilation, floor rising before a +40 kW step, margin monotonicity, validity of candidates against the battery limits.

6.4 Balance — balance.py

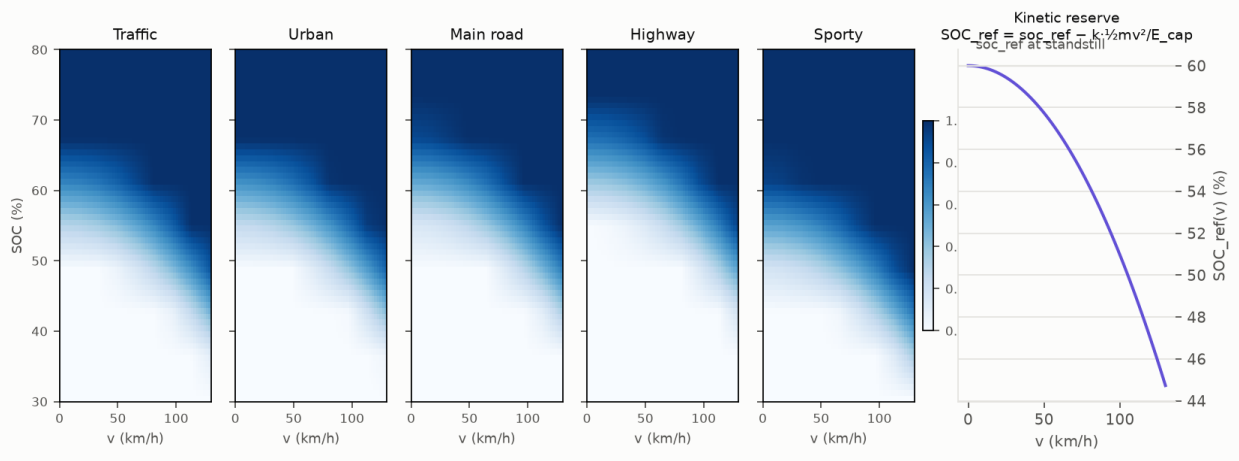


Figure 10 — Battery usage table per driving pattern (left) and the kinetic reserve SOC_ref(v) (right).

Role. Turn the context (speed, SOC, driving pattern) into a co-state λ for the optimiser, through a battery usage ratio and a SOC target.

Algorithm.

8. **Usage ratio** `u ∈ [0, 1] = f(v_norm, SOC_norm, pattern)`, read in a 9 × 9 bilinear table per pattern, built from three explicit effects: a nominal usage per pattern (sporty 0.85, traffic 0.60, urban 0.55, main road 0.45, highway 0.30), the **kinetic reserve** `SOC_ref(v) = soc_ref − k·½·m·v²/E_cap` (k = 0.75), and a proportional anchoring `u = u_nominal + gain·(SOC − SOC_ref(v))/window` with gain 3. A usage of ≈ 0.5 is neutral (λ ≈ λ0); above, the EMS draws on the battery; below, it recharges it.
9. **SOC target**: the reachable SOC range at the end of the horizon is computed from the battery limits, and the target is the point of that range weighted by the usage (usage 1 → bottom of the range, usage 0 → top).
10. **Co-state**: `λ = λ0 + Kp·(SOC_target − SOC)`, saturated in [0.5, 5], with λ0 = 1.51 (the bisection value of the offline ECMS on WLTC) and Kp = 5.

The kinetic reserve. At 125 km/h the vehicle stores ½·m·v² ≈ 1.08 MJ of kinetic energy, of which about 80 % comes back into the battery at the next braking: 15 points of SOC on a 1.6 kWh battery. The reference SOC is therefore lowered with the square of the speed, so that the battery has room for it. This is the classic rule of series hybrids and it is what gives charge-sustaining *without knowing the end of the cycle*: with the reserve disabled, WLTC ends at +7.8 points instead of −0.55.

Tests (`tests/test_balance_optimizer.py`): table shape and bounds, monotonic in SOC and in speed, nominal value at standstill and reference SOC, target inside the reachable range, saturation of λ on both sides.

6.5 Optimizer — optimizer.py

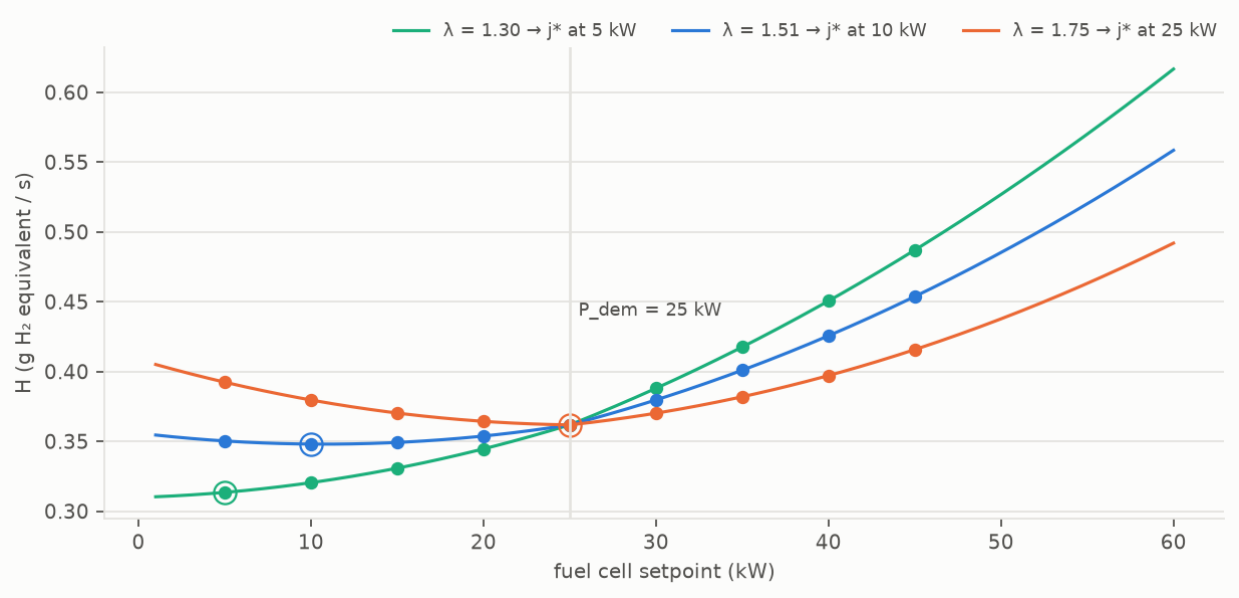


Figure 11 — Hamiltonian of the candidates for three values of λ at a 25 kW demand. A higher λ makes the battery "expensive" and moves the minimum towards the fuel cell.

Role. Pick the candidate that minimises the Hamiltonian of Pontryagin's minimum principle:

`H_j = ṁ_H2,j − λ · k_h · dSOC_j/dt`

with `ṁ_H2,j = P_H2(P_fc,j)/LHV` and `dSOC_j/dt` from the battery model for `P_bat,j = P_dem − P_fc,j`.

The homogenisation factor. `ṁ_H2` (kg/s) and `dSOC/dt` (1/s) are not homogeneous (pitfall 5). The factor `k_h = E_cap / LHV` (in kg) converts a SOC variation into the mass of hydrogen holding the same energy. With it, `λ·k_h·dSOC/dt = −λ·P_int/LHV`, and λ is a **dimensionless equivalence factor**, the inverse of an average efficiency of recharging the battery through the fuel cell (order of magnitude 1/0.6 ≈ 1.5). This is what makes the tuning of λ0 and Kp interpretable. The continuous optimum in discharge is `P* = (λ/η_dis − b)/(2c)`: about 64 kW per unit of λ, which is why the co-state loop gains are small.

Penalties are a pass-through in v1 (`H' = H`), with the hook kept for v2 (ramp and violation penalties). The argmin is taken over the valid candidates; if none is valid, over those with the smallest excess.

Tests (`tests/test_balance_optimizer.py`): `k_h = E_cap/LHV`, Hamiltonian equal to the hydrogen rate at λ = 0, numerical minimum at the analytical `P*`, monotonicity in λ, argmin restricted to valid candidates, `FC_Power_cmd + BAT_Power_cmd = P_dem`.

6.6 Strategies — strategies.py

All strategies implement `Strategy.step(obs: EMSObservation) → EMSCommand` and `reset()`, and share the candidate block. `RulesBaseline` is fifteen lines (low-pass filter plus a three-mode SOC hysteresis); `ECMSConst` calls the optimiser with a fixed λ; `IEMS` is the full pipeline. `calibrate_ecms_const(cycle, cfg)` runs a bisection on λ ∈ [1, 4] until `|SOC(T) − SOC(0)| < 0.5 point`; the final SOC is monotonic in λ, which makes the bisection well-posed thanks to the convex fuel cell model. `make_strategy(name, cfg)` instantiates from the registry.

Tests (`tests/test_strategies_metrics.py`): charge-sustaining of the iEMS on WLTC, zero violation on the five cycles, iEMS ≤ baseline on at least four cycles, calibration convergence and monotonicity, hysteresis of the baseline on hand-built observations.

6.7 Simulator — simulator.py

The loop runs at the model step. For each EMS step it builds the observation, calls the strategy, then applies the held command for `substeps` model steps: the fuel cell moves towards the setpoint under its ramp and within its bounds; the battery closes the bus within its limits; what cannot be closed is logged as `P_friction` (regeneration diverted to the brakes, bounded by the braking power), `P_spill` (fuel cell surplus the battery cannot absorb) or `P_unmet` (traction not served). The bus residual `|P_fc + P_bat − (P_dem + P_friction + P_spill − P_unmet)|` is checked at every step and the run aborts above 1 µW. SOC and hydrogen mass are integrated and logged; the diagnostics of the strategy (λ, SOC target, pattern, chosen candidate, bounds) are logged at the EMS rate in `ems_info`.

Tests (`tests/test_simulator.py`): energy conservation (relative error < 1e-6, bus residual < 1 mW), command held and ramp respected, SOC inside the window, bit-for-bit determinism, WLTC in under 5 s, regenerative braking diverted to friction when the battery is full, a custom predictor accepted through the protocol.

6.8 Metrics — metrics.py

`compute_metrics(result)` returns consumption in g and g/100 km, raw and SOC-corrected; ΔSOC in points; six violation counters (SOC window, fuel cell bounds, applied ramp, commanded ramp, battery discharge, battery charge); and a global energy balance (sources = fuel cell energy + destocked energy; uses = demand served + friction + spill + battery losses) with its relative error. Numerical residuals below 1e-9 are displayed as such, so that reports are reproducible between processors.

7 Verification and validation

In two sentences. Three levels protect the software: unit and integration tests for each brick, six quantified acceptance criteria for the whole, and a cross-verified compliance review against the specification. Judging any change is a three-command ritual.

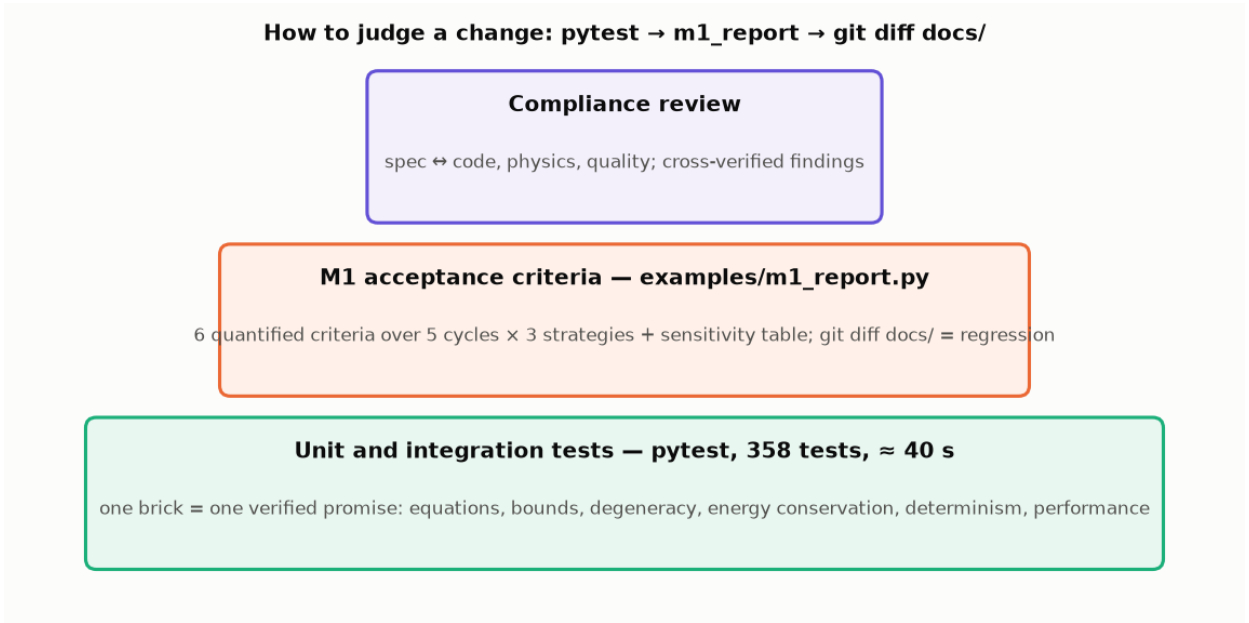


Figure 12 — The validation pyramid.

7.1 Automated tests

`pytest` runs 358 tests in about 40 s (12 s on an Apple Silicon laptop). They are the formal expression of the requirements of the specification: energy conservation, candidate bounds, degeneracy and fallback vector, monotonicity of the battery limits with SOC, charge-sustaining, determinism with fixed seeds, performance. Each test file matches a module group (chapter 3.1); each test has a one-line docstring stating the expected property, and explicit tolerances declared as module constants.

7.2 Acceptance criteria and the reference report

`examples/m1_report.py` runs the full campaign (5 cycles × 3 strategies, ECMS calibrated with exact prediction) and checks the six M1 criteria:

#	Criterion	Result
1	Energy balance closed to < 1 % on WLTC	< 1e-9 %
2	Zero bound violation (SOC, powers, ramps) on 5 cycles	0 for all strategies
3	Charge-sustaining iEMS: abs(ΔSOC) < 2 points on WLTC	−0.55 point
4	iEMS consumption ≤ baseline on ≥ 4 cycles out of 5	5 out of 5
5	iEMS within 5 % of the calibrated ECMS	+1.6 % at worst
6	Thesis-style figure generated by script	yes

It writes `docs/m1_report.md`, the comparison tables and the figures, and returns a non-zero exit code if a criterion fails. Timing values and floating-point residuals are kept out of the written files, so that a regenerated report is byte-identical to the committed reference when nothing changed.

7.3 How to judge a change

```
python -m pytest -q                          # every brick still keeps its promise
python examples/m1_report.py --out docs      # the whole still meets the criteria
git diff --stat docs/                        # what moved, and only that
```

An empty diff means the behaviour is unchanged; otherwise every differing line of `docs/m1_report.md` is a real change of behaviour to explain in the commit message.

7.4 Known limits of version 1

- The WLTC is a structural reconstruction, not the official table (chapter 4.4).
- The predictor is a noisy oracle; a learned predictor will have a different error structure.
- Parameters are generic (`TODO-thèse`); the numbers above are relative comparisons, not absolute predictions for a given vehicle.
- Near `soc_max`, the fuel cell idle power (1 kW) exceeds the auxiliaries (500 W) and cannot be absorbed at standstill: a run started inside the upper derating band shows structural spill. The iEMS keeps away from it by design; the dashboard of M4 should bound the initial SOC.
- Admissibility margins are sized for the default prediction error; at three times that error, violations reappear.

8 Using the package

In two sentences. Three commands install and verify the package; three scripts cover comparison, acceptance and signal plotting; the Python API is five lines.

8.1 Installation

```
python3 -m venv .venv && source .venv/bin/activate     # Windows: .venv\Scripts\activate
pip install -e ".[dev]"
python -m pytest -q && python examples/m1_report.py
```

Python ≥ 3.11; dependencies: numpy, scipy, pydantic, matplotlib. No MATLAB, no network. On a Windows console in code page 1252, run Python with `PYTHONUTF8=1`.

8.2 The three scripts

Script	What it does	Output
`examples/compare_strategies.py [--cycles all] [--soc0 0.35]`	Comparison table and thesis-style figures	`examples/output/`
`examples/m1_report.py [--out docs]`	Six acceptance criteria, sensitivity table, reference figures	`examples/output/` or `docs/`
`examples/plot_run.py <cycle> <strategies…> [--soc0] [--kp] [--lambda0] [--no-noise] [--save]`	Six-panel signal view, several strategies overlaid	window or PNG

8.3 The Python API

```
from iems.config import IEMSConfig
from iems.simulator import run
from iems.metrics import compute_metrics

cfg = IEMSConfig().with_updates(ems={"Kp": 2.0}, soc0=0.45)
res = run("wltc", "iems", cfg)          # SimResult: t, v, P_dem, P_fc, P_bat, soc, m_h2, ems_info…
print(compute_metrics(res).row())
```

`with_updates` derives a new validated configuration without touching the files; `run` accepts any cycle of the catalogue and any strategy of the registry; `SimResult` carries the model-rate signals and the EMS-rate diagnostics.

8.4 Typical experiments

- **Disable the kinetic reserve**: `ems={"kinetic_recovery": 0.0}` — WLTC ends at +7.8 points; criterion 3 fails.
- **Harder prediction**: `ems={"predictor_noise": 0.09, "predictor_time_warp": 0.09}` — violations reappear.
- **A faster fuel cell**: `fuel_cell={"ramp_W_per_s": 20e3}` — less anticipation needed, smaller floor.
- **A new cycle**: build `Cycle(name, t, v)` from any speed-over-time CSV and register it in `models/cycles.py`.
- **A new strategy**: a class with `step(obs)` returning `EMSCommand`, added to `STRATEGIES`.

9 Roadmap, glossary, references

9.1 Roadmap

Milestone	Content	Exit criterion
M1 — Simulator	models, three strategies, metrics, figures, tests	done: 6/6 criteria
M2 — API	FastAPI (`POST /simulate`, `POST /compare`, `GET /strategies`, `GET /cycles`), Docker image, API tests	`docker run` → JSON result in under 5 minutes
M3 — Agent	tools for an LLM agent (function calling / MCP): run, compare, generate cycle, explain result	a natural-language request triggers runs and explains the figures
M4 — Customer case	CSV fleet ingestion, Streamlit dashboard, one-page architecture note	a non-specialist understands the value in five minutes
v2 (optional)	LSTM predictor (torch), fuzzy c-means recogniser and fuzzy balance (scikit-fuzzy), thesis parameters, official WLTC table	same interfaces

9.2 Glossary

Term	Meaning
SOC	State of charge of the battery, 0 to 1 (or in %).
Charge-sustaining	The SOC ends a representative trip close to where it started.
EMS	Energy management strategy: the controller that splits the power demand between sources.
PMP, Hamiltonian	Pontryagin's minimum principle; the Hamiltonian is the instantaneous cost whose minimisation gives the optimal control, here hydrogen flow plus the weighted SOC variation.
ECMS	Equivalent consumption minimisation strategy: a practical form of PMP where λ is an equivalence factor between battery energy and hydrogen.
λ, co-state	The weight of battery energy in the Hamiltonian; dimensionless here thanks to k_h.
k_h	Homogenisation factor E_cap/LHV (kg): converts a SOC variation into an equivalent hydrogen mass.
Hp	Prediction horizon of the EMS (10 s).
Ts_model, Ts_ems	Model step (0.1 s) and EMS step (1 s).
DPT, DPR	Driving pattern tools; driving pattern recognition.
Candidate, fallback vector	One of nine possible fuel cell setpoints; the degenerate vector used when the range collapses.
Kinetic reserve	SOC left free for the energy that will come back at the next braking, proportional to ½·m·v².
WLTC	Worldwide harmonised light vehicles test cycle, class 3b (the homologation cycle).
LHV	Lower heating value of hydrogen, 120 MJ/kg.

9.3 References

- Pontryagin's minimum principle and ECMS: Paganelli et al., *Equivalent consumption minimization strategy for parallel hybrid powertrains*, IEEE VTC 2002; Onori, Serrao, Rizzoni, *Hybrid Electric Vehicles: Energy Management Strategies*, Springer 2016.
- Adaptive ECMS (A-ECMS) with SOC feedback: Musardo, Rizzoni, Staccia, *A-ECMS: An adaptive algorithm for hybrid electric vehicle energy management*, CDC 2005.
- Speed-dependent SOC reference (kinetic reserve) in series and power-split hybrids: Guzzella, Sciarretta, *Vehicle Propulsion Systems*, Springer, 3rd ed.
- WLTP / WLTC class 3b phase structure: UNECE Global Technical Regulation No. 15.
- Author's thesis: integrated EMS for fuel cell hybrid vehicles (online PMP, fuzzy rules, driving pattern recognition, LSTM speed prediction), ESTACA / Université Paris-Saclay. Public defence slides.